

ViewModel

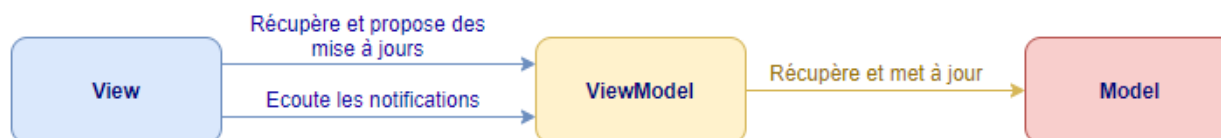
Table des matières

Modèle-vue-vue modèle	2
Création du modèle	2
Création de la vue	2
Création de la vue modèle	5
Test du modèle de vue	7
Exercices	8
Références	8

Modèle-vue-vue modèle

Le modèle-vue-vue modèle (MVVM) est un modèle de conception développé en 2004 par Microsoft. MVVM vise à séparer la logique de présentation d'une application en 3 couches :

- **Modèle**: Le modèle contient les données liés à la logique métier. Il peut s'agir par exemple d'entités issues de bases de données ou encore d'API externes.
- **Vue**: La vue est la description de l'interface graphique, elle fait le lien entre les actions de l'utilisateur et le modèle de vue.
- **Vue modèle**: Le modèle de vue est chargé de transformer et organiser les modèles métiers afin d'exposer les données à afficher par la vue.



Étant un facteur important de maintenabilité, les dépendances entre les différentes couches doivent être optimisés. Le ViewModel notifie les changements qui lui sont appliqués. De cette manière, la View est elle même responsable de se tenir informée des mise à jours du ViewModel par le biais de notifications (événements) et de se mettre à jour au besoin.

Création du modèle

Reprenons l'exemple du chapitre 3: Le convertisseur de pouces à centimètre.

1. Dans le package `ui`, ajoutez une classe (Data Class) appelée `ui/ConvUiState.kt`. Cette classe représente le modèle des données qui seront affichées par l'interface (la vue).

```
1 package com.example.convertisseur2.ui
2
3 data class ConvUiState(
4     var cm: Double = 0.0
5 )
```

Création de la vue

1. Dans le package `ui`, ajoutez un fichier `ui/ConvScreen.kt` qui contient un composant appelée `ConvScreen()`. Ce composant contient la description de l'interface de l'application.

```
1 @OptIn(ExperimentalMaterial3Api::class)
2 @Composable
3 fun ConvScreen () {
4     var entree: String = ""
5     Column(
```

```

6      modifier = Modifier.fillMaxSize(),
7      verticalArrangement = Arrangement.Top,
8      horizontalAlignment = Alignment.CenterHorizontally
9
10     ){
11         Text(
12             text = "Pouce à cm",
13             fontSize = 36.sp,
14             modifier = Modifier.padding(
15                 start = 16.dp,
16                 top = 16.dp,
17                 end = 16.dp,
18                 bottom = 32.dp
19             )
20         )
21
22         TextField(
23             value = entree,
24             onChange = { entree = it },
25             label = { Text(text = "Pouces: ") },
26             trailingIcon = {
27                 Icon(
28                     Icons.Default.Clear,
29                     contentDescription = "clear text",
30                     modifier = Modifier
31                         .clickable {
32                             entree = ""
33                         }
34                 )
35             },
36         )
37         Spacer(Modifier.height(16.dp))
38         Button(onClick = {
39             // Action ici
40         }) {
41             Text(text = "Convertir")
42         }
43         Spacer(Modifier.height(16.dp))
44         Text(
45             text = "Résultat: ",
46             color = Color.Red,
47         )
48     }
49 }

```

2. Le fichier `MainActivity.kt` est maintenant simplement composé de:

```

1 class MainActivity : ComponentActivity() {
2     override fun onCreate(savedInstanceState: Bundle?) {
3         super.onCreate(savedInstanceState)
4         setContent {
5             Convertisseur2Theme {

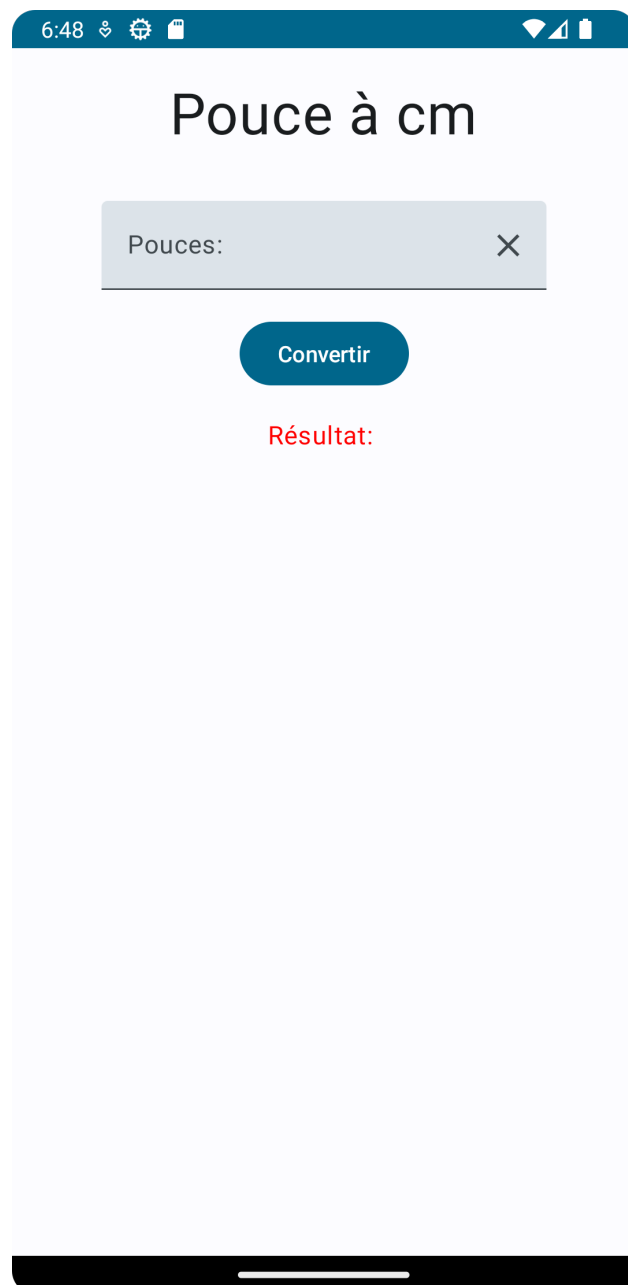
```

```

6         Surface(
7             modifier = Modifier.fillMaxSize(),
8             color = MaterialTheme.colorScheme.background
9         ) {
10             ConvScreen()
11         }
12     }
13 }
14 }
15 }

```

3. Vérifier le fonctionnement de votre interface utilisateur.



Création de la vue modèle

`ui/ConvViewModel.kt` → La logique de l'application

1. Ouvrez le fichier `build.gradle(Module: app)` dans le bloc `dependencies` et ajoutez la dépendance suivante pour `ViewModel`.

```
1 implementation "androidx.lifecycle:lifecycle-viewmodel-compose:2.5.1"
```

2. Dans le package `ui`, créez une classe Kotlin appelé `ConvViewModel`. Vous l'étendrez à partir de la classe `ViewModel`.

```
1 package com.example.convertisseur2.ui.theme
2
3 import androidx.compose.runtime.mutableStateOf
4 import androidx.lifecycle.ViewModel
5 import com.example.convertisseur2.ui.ConvUiState
6 import kotlinx.coroutines.flow.MutableStateFlow
7 import kotlinx.coroutines.flow.StateFlow
8 import kotlinx.coroutines.flow.asStateFlow
9 import kotlinx.coroutines.flow.update
10
11 class ConvViewModel : ViewModel(){ // Héritage
12
13     // L'état de l'interface (UI State)
14     private val _uiState = MutableStateFlow(ConvUiState())
15     val uiState: StateFlow<ConvUiState> = _uiState.asStateFlow()
16
17     // L'information provenant de la zone de texte (Pouce)
18     private var _pouce = mutableStateOf("")
19
20     fun updatePouce(pouce: String){
21         _pouce.value = pouce
22     }
23
24     fun getPouce(): String{
25         return _pouce.value
26     }
27
28     fun resetPouce(){
29         _pouce.value = ""
30     }
31
32     fun convertir(){
33         var pouce = _pouce.value.toDouble()
34         var nvCm = pouce * 2.54
35         _uiState.update{currentState ->
36             currentState.copy(
37                 cm = nvCm
38             )
39         }
```

```

40     }
41 }

```

3. Modifiez la vue (`ui/ConvScreen.kt`) afin de faire le lien avec le modèle de vue.

```

1  ..
2  ..
3  fun ConvScreen (convViewModel: ConvViewModel = viewModel()){
4  *   val convUiState = convViewModel.uiState.collectAsState()*
5      Column(
6          modifier = Modifier.fillMaxSize(),
7          verticalArrangement = Arrangement.Top,
8          horizontalAlignment = Alignment.CenterHorizontally
9      )
10  ){
11      Text(
12          text = "Pouce à cm",
13          fontSize = 36.sp,
14          modifier = Modifier.padding(
15              start = 16.dp,
16              top = 16.dp,
17              end = 16.dp,
18              bottom = 32.dp
19          )
20      )
21
22      TextField(
23          value = convViewModel.getPouce(),
24          onChange = { convViewModel.updatePouce(it) },
25          label = { Text(text = "Pouces: ") },
26          trailingIcon = {
27              Icon(
28                  Icons.Default.Clear,
29                  contentDescription = "clear text",
30                  modifier = Modifier
31                      .clickable {
32                          convViewModel.resetPouce()
33                      }
34              )
35          },
36      )
37      Spacer(Modifier.height(16.dp))
38      Button(onClick = {
39          convViewModel.convertir()
40      }){
41          Text(text = "Convertir")
42      }
43      Spacer(Modifier.height(16.dp))
44      Text(
45          text = "Résultat: ${convUiState.value.cm}",
46          color = Color.Red,
47      )

```

```
48     }  
49 }
```

Portez attention aux lignes 3, 4, 23, 24, 32, 39 et 45.

Test du modèle de vue

Android Studio fournit des outils qui permettent d'automatiser des tests. Ainsi, il est possible de faire des tests qui vérifient automatiquement que la conversion de pouce à centimètre s'effectue correctement.

Dans le paquetage `com.exemple.convertisseur2(test)` modifiez le fichier `ExampleUnitTest.kt`

```
1 package com.exemple.convertisseur2  
2  
3 import com.exemple.convertisseur2.ui.theme.ConvViewModel  
4 import org.junit.Test  
5  
6 import org.junit.Assert.*  
7  
8 class ExampleUnitTest {  
9     private val viewModel = ConvViewModel()  
10    @Test  
11    fun conv_isCorrect() {  
12        viewModel.updatePouce("1.0")  
13        viewModel.convertir()  
14        var currentConvUiState = viewModel.uiState.value  
15        assertEquals(2.54, currentConvUiState.cm, 0.1)  
16    }  
17 }
```

Exercices

Exercice 1

Refaire l'exemple des notes de cours.

Exercice 2

Reprenons l'exercice 2 du chapitre 3: Programmer une application qui permet de calculer l'aire et le périmètre d'un rectangle.

Références

- [ViewModel et l'état dans Compose](#)